

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

Desarrollo Web · II semestre 2026

Laboratorio de seguridad con OpenSSL en PHP

Ricardo Caballero

Grupo 1S3122

ricardo.caballero5@utp.ac.pa

Docente: Ing. Irina Fong

29 de octubre de 2026

1. Introducción y objetivos

La protección de datos exige distinguir confidencialidad, integridad y autenticidad. En este laboratorio se recrean los ejemplos de la docente con formularios para entradas variables, tratamiento de errores y resultados visibles. OpenSSL aporta firma RSA, solicitudes CSR, certificados X.509 y cifrado; Sodium permite comparar una interfaz de cifrado autenticado.

Objetivo general: implementar y comprobar operaciones criptográficas en PHP, documentando sus parámetros, resultados y limitaciones. Objetivos específicos: generar un par RSA de 2048 bits, validar firmas SHA-256, emitir un certificado de prueba, recuperar texto cifrado y demostrar el rechazo de alteraciones.

La ejecución se realizó con PHP 8.3.36, OpenSSL 3.0.22 y Sodium 1.0.22. Se trabajó en localhost con datos ficticios. Ningún certificado generado en esta práctica sustituye un certificado de una autoridad de confianza.

Código y evidencias: https://github.com/ricardo-caballero2663/ricardo-caballero-laravel

Componente	Implementación y resultado
firma7	Firma RSA/SHA-256; prueba válida y prueba con mensaje alterado.
firmaOtra	Formulario de identidad de prueba; genera CSR y certificado autofirmado.
FirmaMensaje	Usa la clave privada y el certificado de firmaOtra en la misma sesión.
Encriptacion	Reproduce AES-128-CBC y añade una comparación autenticada AES-256-GCM.
Sodium	Secretbox: recuperación correcta y rechazo de cifrado manipulado.

2. Implementación y decisiones técnicas

Los ejemplos de referencia son firma7.php, firmaOtra.php, FirmarMesaje.php y Encriptacion.php. Se conserva su propósito y se reorganiza la implementación en crypto.php y public/index.php. FirmaMensaje corrige el nombre escrito como FirmarMesaje en el repositorio original. La navegación presenta los nombres solicitados por la guía.

Se reemplazó la ruta absoluta de OpenSSL por __DIR__/openssl.cnf. Se distinguen la CSR y el certificado: la solicitud empieza con BEGIN CERTIFICATE REQUEST; el certificado con BEGIN CERTIFICATE. No se reutilizan las llaves que ya estaban publicadas en el repositorio de referencia.

Los formularios validan mensajes no vacíos y longitudes; escapan la salida HTML y usan un token CSRF. Las claves de firmaOtra se guardan temporalmente en la sesión del servidor. El directorio público contiene solo el punto de entrada. Las claves mostradas en las capturas son exclusivamente material de demostración, sin uso fuera del laboratorio.

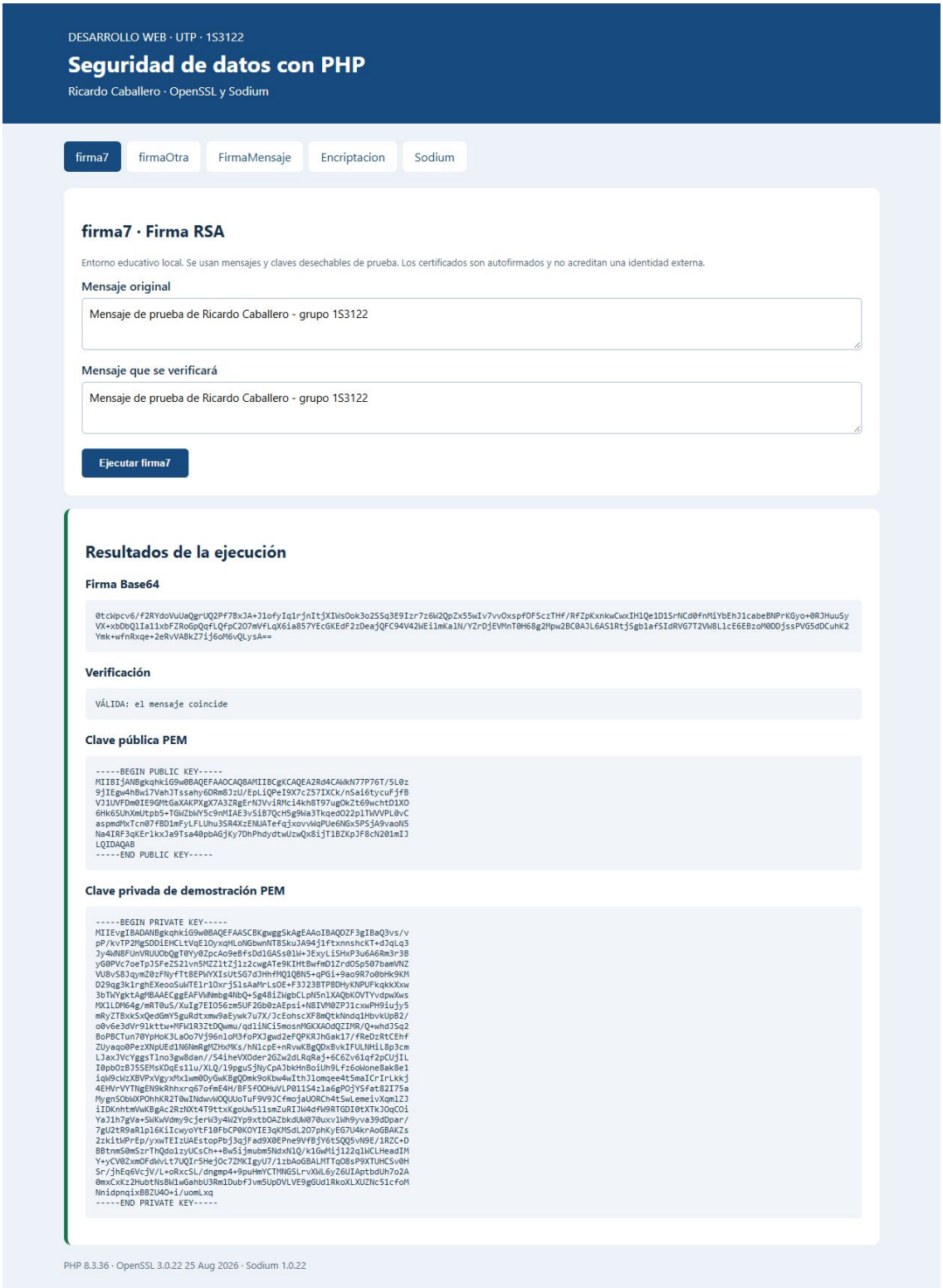
El cifrado CBC original permite recuperar el mensaje, pero por sí solo no prueba integridad. Se añadió AES-GCM con nonce de 12 bytes y etiqueta autenticadora. La prueba altera un byte del cifrado y confirma que el descifrado autenticado lo rechace. No se confunde Base64 con cifrado.

Las contraseñas de usuarios no deben almacenarse mediante cifrado reversible: requieren hashing apropiado, por ejemplo Argon2id. Las claves criptográficas deben mantenerse separadas del cifrado. Para datos en tránsito se requiere TLS; este servidor local HTTP sirve únicamente para la práctica.

Resultado de validación: 11 pruebas automatizadas aprobadas y seis capturas de ejecución. También se verificó que el certificado corresponda a la clave privada y que se rechacen entradas vacías.

3. Evidencia: firma7

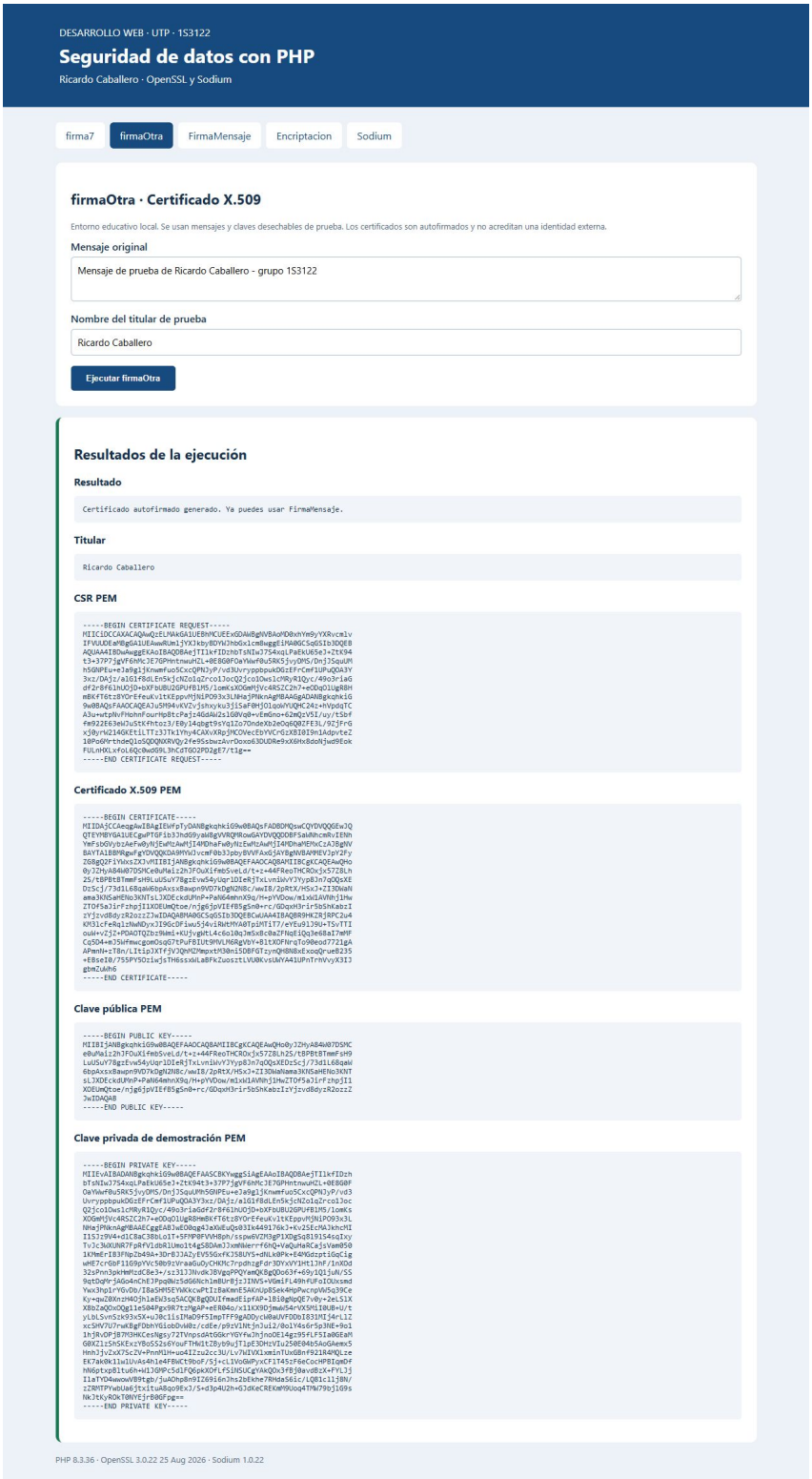
El formulario recibe dos mensajes iguales. La verificación devuelve VÁLIDA y se muestran firma y par de claves generado.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

4. Evidencia: firmaOtra

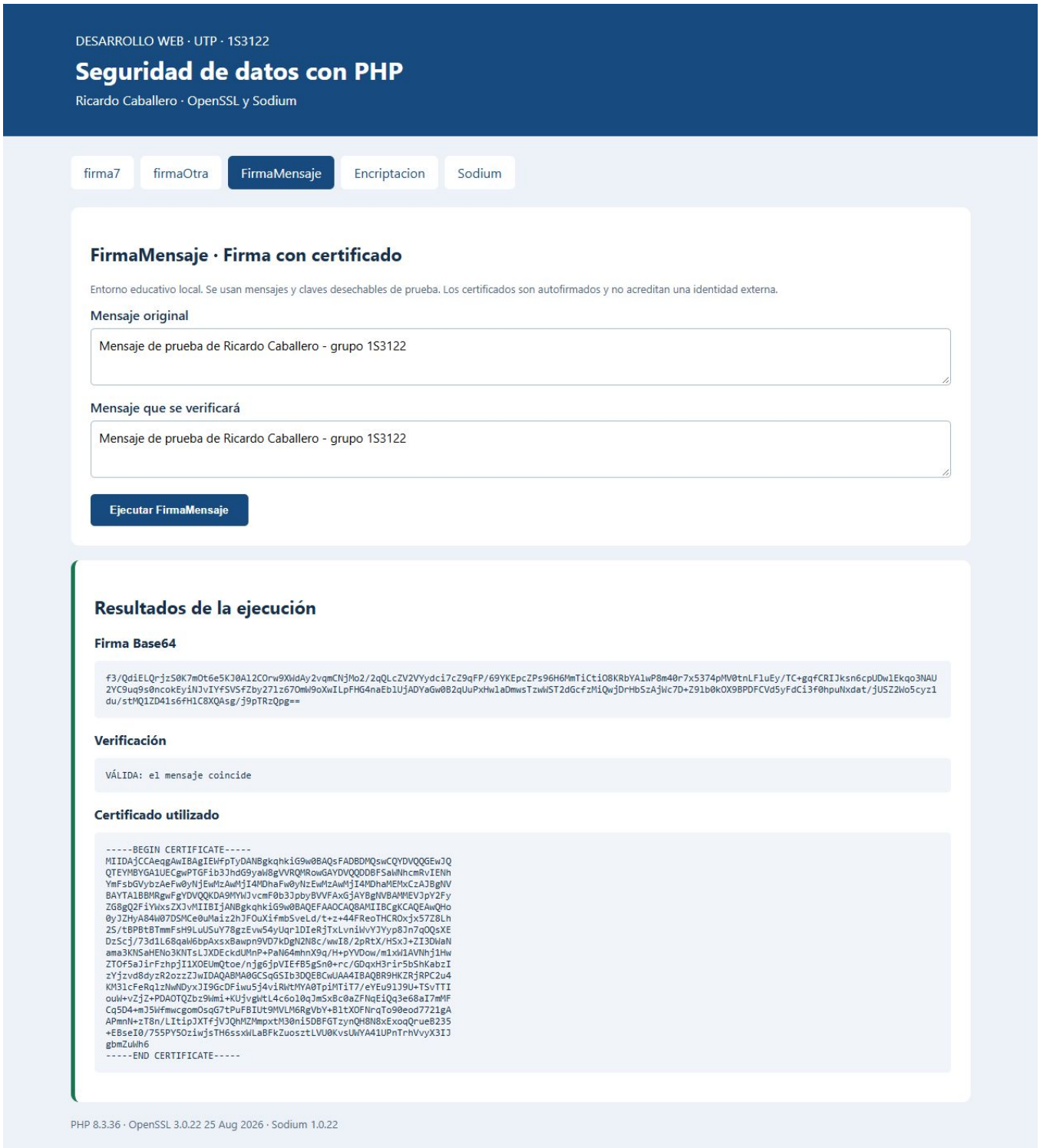
El nombre ingresado se incorpora al certificado de prueba. La pantalla muestra la CSR, el X.509 y las claves generadas.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

5. Evidencia: FirmaMensaje

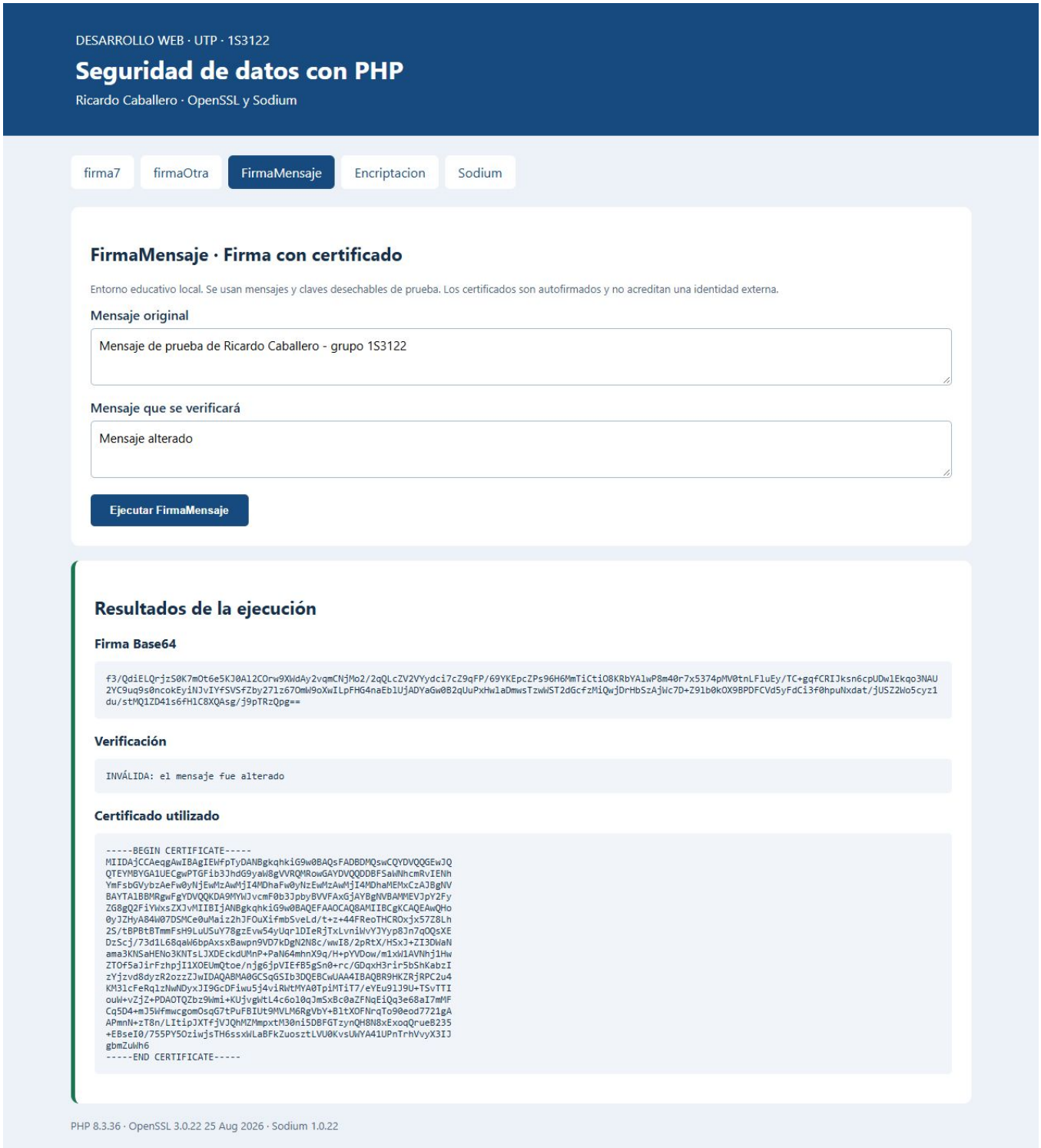
La firma se verifica con el certificado creado anteriormente en la misma sesión. Resultado: VÁLIDA.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

6. Evidencia: alteración

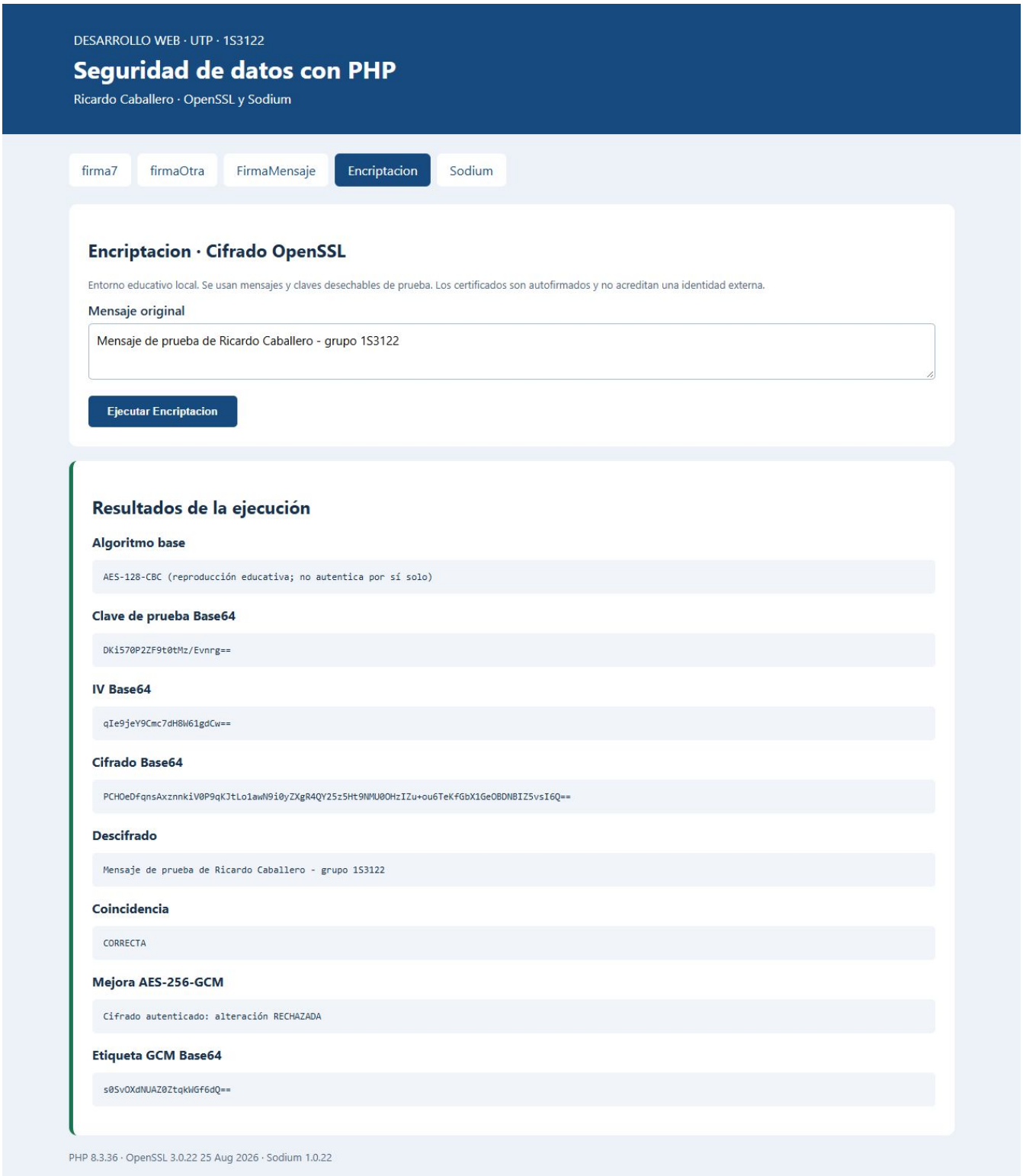
Al sustituir el texto de verificación por Mensaje alterado, la firma pasa a INVÁLIDA. La firma no oculta el mensaje; permite detectar cambios.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

7. Evidencia: cifrado OpenSSL

AES-CBC recupera el texto original. La comparación adicional con AES-GCM rechaza el cifrado alterado.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

8. Evidencia: Sodium

Secretbox recupera el mensaje y rechaza la alteración. El nonce y el cifrado se muestran en Base64 para facilitar la inspección.



Captura de ejecución local del 29 de octubre de 2026. Los valores criptográficos son desechables y cambian entre ejecuciones.

9. Documentación técnica de funciones

Función	openssl_pkey_new / openssl_pkey_export
Propósito	Generar RSA y exportar PEM
Parámetros	Arreglo con RSA, 2048 bits y archivo openssl.cnf; clave, variable PEM por referencia, contraseña opcional.
Retorno	Objeto OpenSSLAsymmetricKey o false; export devuelve bool.
Proceso	Genera dos claves relacionadas. La privada se conserva solo en la sesión de demostración.

Función	openssl_pkey_get_details / openssl_pkey_get_private
Propósito	Obtener clave pública o cargar privada
Parámetros	Objeto de clave; cadena PEM privada.
Retorno	Arreglo con key/bits/type o false; objeto de clave o false.
Proceso	Permite separar verificación pública y firma privada.

Función	openssl_sign
Propósito	Firmar mensaje
Parámetros	Datos, firma binaria por referencia, clave privada y OPENSSL_ALGO_SHA256.
Retorno	bool; firma se escribe en el parámetro de salida.
Proceso	RSA firma un resumen SHA-256. No cifra el texto ni lo vuelve confidencial.

Documentación técnica (continuación)

Función	openssl_verify
Propósito	Verificar firma
Parámetros	Mensaje, firma binaria, clave pública o certificado, algoritmo SHA-256.
Retorno	1 válida; 0 inválida; -1 o false si hay error.
Proceso	Comprueba correspondencia entre mensaje y firma. Se distinguen alteración y fallo técnico.

Función	openssl_csr_new / openssl_csr_export
Propósito	Crear solicitud de certificado
Parámetros	DN (país, organización, nombre), clave privada, configuración; CSR y salida PEM.
Retorno	OpenSSLCertificateSigningRequest o false; export devuelve bool.
Proceso	La CSR contiene identidad declarada y clave pública, firmadas con la privada.

Función	openssl_csr_sign / openssl_x509_export
Propósito	Emitir certificado de prueba
Parámetros	CSR, certificado emisor null, clave privada, 365 días, configuración, serie; certificado y salida PEM.
Retorno	OpenSSLCertificate o false; export devuelve bool.
Proceso	Se crea un X.509 autofirmado. La confianza externa requiere una autoridad o mecanismo de confianza adicional.

Documentación técnica (continuación)

Función	openssl_encrypt / openssl_decrypt
Propósito	Cifrar y recuperar datos
Parámetros	Texto/cifrado, algoritmo, clave, opciones, IV; en GCM etiqueta y datos asociados opcionales.
Retorno	Cadena binaria con OPENSSL_RAW_DATA o false.
Proceso	CBC usa relleno y exige protección de integridad adicional; GCM incorpora etiqueta autenticadora.

Función	random_bytes / openssl_cipher_iv_length
Propósito	Generar aleatoriedad y medir IV
Parámetros	Cantidad de bytes; nombre del cifrador.
Retorno	Cadena aleatoria; longitud entera o false.
Proceso	Evita IV constantes. La clave y el IV se generan con tamaños adecuados al algoritmo.

Función	sodium_crypto_secretbox_keygen
Propósito	Generar clave simétrica Sodium
Parámetros	Sin parámetros.
Retorno	32 bytes aleatorios.
Proceso	Produce la clave requerida por secretbox; no se reutiliza para otros fines.

Documentación técnica (continuación)

Función	sodium_crypto_secretbox
Propósito	Cifrado autenticado
Parámetros	Mensaje, nonce único de 24 bytes y clave de 32 bytes.
Retorno	Cadena con cifrado y autenticador.
Proceso	XSalsa20 cifra y Poly1305 autentica. El nonce puede acompañar al cifrado; la clave debe permanecer secreta.

Función	sodium_crypto_secretbox_open / sodium_memzero
Propósito	Verificar, descifrar y limpiar clave
Parámetros	Cifrado, nonce y clave; variable sensible por referencia.
Retorno	Texto o false si falla autenticación; memzero devuelve void.
Proceso	No se usa el texto si falla la autenticación. La limpieza reduce la permanencia de la clave en memoria.

Función	base64_encode / base64_decode
Propósito	Representar binarios como texto
Parámetros	Cadena binaria; cadena Base64 y strict=true al decodificar entradas externas.
Retorno	Cadena; decode puede devolver false.
Proceso	Base64 es codificación, no cifrado. Se usa para mostrar firmas, nonce y resultados.

10. Investigación: Libsodium

Libsodium ofrece primitivas criptográficas de alto nivel. En PHP la extensión existe desde la versión 7.2, aunque su disponibilidad efectiva se comprueba en cada instalación. Sus constantes y funciones de generación de claves ayudan a respetar tamaños y formatos; el desarrollador sigue siendo responsable de custodiar la clave y no repetir el nonce con la misma clave.

Operación	Algoritmos y utilidad
Cifrado simétrico	secretbox combina XSalsa20 y Poly1305. Las interfaces AEAD permiten XChaCha20-Poly1305 y datos asociados.
Cifrado asimétrico	crypto_box utiliza claves públicas/privadas para compartir mensajes autenticados; se apoya en Curve25519 y primitivas simétricas.
Firmas	Ed25519 permite firmar y verificar mensajes con claves separadas. Una firma no cifra el contenido.
Hashing	BLAKE2b para hashing general; Argon2id para derivación y protección de contraseñas con costo de memoria.
Rendimiento	Las primitivas están implementadas en una biblioteca nativa y evitan diseñar combinaciones manuales. La velocidad relativa depende de CPU, tamaño y operación; no se realizó un benchmark comparativo.

En el ejemplo implementado se genera una clave de 32 bytes y un nonce aleatorio de 24 bytes. secretbox devuelve el cifrado con su autenticador. secretbox_open comprueba la autenticidad antes de devolver el texto; ante una alteración devuelve false. La captura y las pruebas demuestran ambos caminos.

OpenSSL sigue siendo apropiado cuando se requiere interoperabilidad RSA, CSR o X.509. Sodium simplifica operaciones de aplicación y reduce decisiones peligrosas, pero no elimina la necesidad de una correcta gestión de secretos.

11. Ejemplo ejecutable y conclusiones

```
$key = sodium_crypto_secretbox_keygen();
$nonce = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$cipher = sodium_crypto_secretbox($mensaje, $nonce, $key);
$plain = sodium_crypto_secretbox_open($cipher, $nonce, $key);
if ($plain === false) {
    throw new RuntimeException('Autenticacion fallida');
}
sodium_memzero($key);
```

El código completo incluye formulario, manejo de errores y una prueba de manipulación. Los archivos tests.php y pruebas.txt permiten reproducir la validación. No se guardan llaves privadas operativas en el repositorio; se generan al ejecutar la aplicación.

La comparación de resultados permite concluir que cifrar y firmar resuelven problemas diferentes. Recuperar un texto cifrado no basta para demostrar que nadie lo alteró. Las pruebas negativas hacen visible esta diferencia: RSA, GCM y secretbox rechazan las modificaciones según su mecanismo de verificación.

La práctica también muestra que un certificado autofirmado es útil para experimentar con formatos y operaciones, pero no valida por sí mismo la identidad del titular ante terceros. La protección final depende tanto del algoritmo como de la generación, almacenamiento, rotación y acceso a las claves.

Como criterio de diseño para futuras aplicaciones, conviene reducir los datos sensibles almacenados, usar primitivas documentadas y probar errores además del caso exitoso. El resultado más útil del ejercicio es poder explicar qué garantía aporta cada operación y qué aspectos quedan fuera de ella.

Referencias consultadas el 29 de octubre de 2026

Repositorio de la docente - https://github.com/Salomon2514/SeguridadOpenSSL

PHP OpenSSL - https://www.php.net/manual/en/book.openssl.php

Firma digital en PHP - https://www.php.net/manual/en/function.openssl-sign.php

Cifrado en PHP - https://www.php.net/manual/en/function.openssl-encrypt.php

PHP Sodium - https://www.php.net/manual/en/book.sodium.php

Secretbox - https://doc.libsodium.org/secret-key_cryptography/secretbox

Firmas Ed25519 - https://doc.libsodium.org/public-key_cryptography/public-key_signatures

Hashing de contraseñas - https://doc.libsodium.org/password_hashing

12. Paso a paso de ejecución

1. Instalar PHP 8.2 o superior. Comprobar con php -v y php -m que estén habilitadas openssl y sodium. En Windows, habilitar extension=openssl y extension=sodium en el php.ini correspondiente y reiniciar el servidor.

2. Abrir una terminal dentro de aplicacion. El archivo openssl.cnf incluido evita depender de una ruta fija como C:\OpenSSL-Win64. No se requiere MySQL.

3. Ejecutar php -S 127.0.0.1:18085 -t public. Abrir http://127.0.0.1:18085 en el navegador. El servidor de desarrollo se limita al equipo local.

4. En firma7, escribir un mensaje en ambos campos y pulsar Ejecutar firma7. Debe aparecer VÁLIDA. Cambiar el campo de verificación y repetir para obtener INVÁLIDA.

5. En firmaOtra, escribir el nombre del titular de prueba y ejecutar. Revisar la CSR, el certificado X.509 y las claves PEM generadas. Mantener la misma sesión del navegador.

6. Entrar a FirmaMensaje y ejecutar con textos iguales. Se usa la clave y el certificado creados en el paso anterior. Cambiar el mensaje que se verificará y comprobar que la firma sea rechazada.

7. En Encriptacion, ingresar texto y ejecutar. Comparar el descifrado con el original. Revisar IV, cifrado Base64 y el rechazo de alteración en la prueba adicional AES-256-GCM.

8. En Sodium, ingresar texto y ejecutar. Revisar nonce, cifrado, recuperación del mensaje y rechazo de una modificación de un byte.

9. Ejecutar php tests.php en la carpeta aplicacion. Deben aprobarse 11 pruebas. Si sodium no está cargado, usar php -d extension=sodium tests.php tras verificar que la extensión esté instalada.

10. Cerrar el servidor con Ctrl+C. Las claves de demostración no deben reutilizarse. En producción se requieren gestión de secretos, HTTPS, controles de acceso y almacenamiento seguro.